Smart Blood & Organ Donation Network

Application and Technical Report

Prepared: 26 October 2026 Project type: Java full-stack web application Repository reviewed: `Orgain_Donation`

Executive summary

The application is a small web portal backed by a Spring Boot REST API. A browser page built with HTML, CSS and JavaScript sends JSON requests to the API. The API uses Spring Data JPA to read and write user, blood request, organ preference, donation history and donor notification records. Both the normal and `dev` profiles use MySQL; the `dev` profile only enables sample data and local email settings.

The application uses both seeded and user-submitted data. In the `dev` profile, a startup seeder inserts a demo dataset into MySQL when the database has no users. Portal forms also create and update records through API endpoints. Demo and portal-created data remain in MySQL when the application stops. The default profile uses the same MySQL database but does not seed demo records.

Donor matching is implemented as deterministic rules: blood-group compatibility, donor availability, a 90-day interval since the last donation, and approximate straight-line distance. The project does not use a machine-learning model. Email sending is attempted, but the development profile points to localhost SMTP and the default profile has blank mail credentials; a failed send leaves a notification record marked pending.

This remains a demonstration/MVP rather than a production-ready medical service. Authentication now uses server-side sessions, explicit role checks and ownership checks. Production still requires HTTPS with secure cookies enabled, a shared rate limiter for multiple application instances, review of medical workflows, and managed database migrations.

1. Application purpose and users

The portal models four user roles:

- Donor: maintains a profile and can submit organ-donation consent.
- Patient: creates an emergency blood request and sees matching results.
- Hospital: creates a request on behalf of a hospital and can initiate donor matching.
- Admin: views aggregate dashboard counts and can use the organ preference verification endpoint.

The visible frontend is a single-page portal in `frontend/index.html`, styled by `frontend/assets/css/styles.css` and implemented by `frontend/assets/js/main.js`. The current repository does not contain the separate role-specific page folders described by some documentation; the actual UI panels and forms are in the single index page.

2. Architecture and request flow

```
Browser (HTML/CSS/JavaScript)
        | JSON over HTTP; API_BASE = http://localhost:8080/api
        v
Spring Boot REST controllers
        v
Service layer (business rules and workflow)
        v
Spring Data JPA repositories
        v
MySQL database (all profiles)
```

The controller layer exposes REST routes. Services validate and coordinate work. Repositories persist JPA entities. Hibernate creates or updates tables based on entity mappings because both profiles configure `spring.jpa.hibernate.ddl-auto=update`. `database/schema.sql` documents a corresponding MySQL schema, but the app configuration does not show a script-based migration or automatic execution of that schema file.

The frontend uses `fetch()` to call the API, sends JSON, includes the server session cookie, and sends a CSRF token on state-changing requests. CORS origins are configurable through `APP_CORS_ORIGINS`. After a reload the frontend calls `/api/auth/me` to restore the login state; user identity is not stored in `localStorage`.

3. Main technical workflows

3.1 Registration and login

`POST /api/auth/register` accepts profile fields such as name, email, password, role, blood group, location coordinates, donation flags and last donation date. The service checks for required email/password and duplicate email, then BCrypt-encodes the password before storing it. Public registration defaults to `DONOR` and permits only `DONOR` or `PATIENT`; `ADMIN` and `HOSPITAL` are rejected. Administrative and hospital accounts must be provisioned outside public registration. The response omits the password hash.

`POST /api/auth/login` authenticates through Spring Security's database-backed provider and BCrypt. On success it creates or rotates a server-side HTTP session and returns a safe profile. Failed logins return `401`; an in-memory IP limiter returns `429` after the configured threshold. `GET /api/auth/me` returns the authenticated profile. `POST /api/auth/logout` requires CSRF protection and invalidates the session. Session timeout, cookie security and rate-limit thresholds are environment-configurable.

3.2 Blood request, matching and notification

The portal submits a new request to `POST /api/blood-requests`. The service stores the requester, contact details, location, blood group, units, urgency and notes. When using the portal form, JavaScript immediately calls `POST /api/blood-requests/{id}/match` after creating the request.

The matching service:

- Loads donors marked as blood donors and available.
- Filters to blood groups compatible with the recipient using a fixed compatibility map.
- Excludes donors whose last donation was less than 90 days ago. A donor with no recorded last donation date passes this rule.
- Computes straight-line distance with the Haversine formula using request and donor latitude/longitude.
- Computes a simple score: `max(10, 100 - round(distanceKm))`, adding 10 for a donor with no recorded last donation date.
- Sorts by score descending and then distance ascending, and returns at most 10 matches.

This score is a simple ranking heuristic, not an AI or machine-learning score. Haversine distance is not driving distance; results depend on the accuracy of stored coordinates. The rule does not use units required or urgency to change donor eligibility or ranking.

For each match, the notification service creates a `DonorNotification` row, sets the channel to `EMAIL`, and attempts to send a simple email. A successful send is recorded as `SENT`; a runtime send failure is recorded as `PENDING`. The blood request status is then set to `NOTIFIED` even if email delivery failed. The portal displays the returned matches. `POST /api/blood-requests/{id}/fulfill` can set request status to `FULFILLED`.

3.3 Organ donation preference

`POST /api/organs/preferences` saves or updates the authenticated donor's preference, including organs and consent statement, and sets that user's `organDonor` flag to true. Donors may update only their own preference. Only an administrator may verify or list preferences. Responses use a DTO and do not serialize linked user accounts.

3.4 Donation history

`POST /api/donations` creates a donation history entry for an existing donor. If no donation date is supplied, it uses the current date. It also updates the donor's `lastDonationDate`, which affects future eligibility matching. `GET /api/donations/donor/{donorId}` lists that donor's history. These API capabilities exist in the backend; the current portal JavaScript does not wire a donation-history form or display.

3.5 Admin dashboard

`GET /api/admin/dashboard` returns counts for all users, blood donors, organ donors, open or notified requests, and verified organ preferences. The portal requests this endpoint to refresh its dashboard values.

4. Data model

Entity/table	Purpose and important links
`UserAccount` / `user_account`	Login/profile, role, blood group, contact/location, donor flags, availability and last donation date. Email is unique.
`BloodRequest` / `blood_request`	Requester and hospital contact, location, required blood group, units, urgency, notes and status.
`DonorNotification` / `donor_notification`	Links a donor and request; records channel, message, status and time.
`DonationHistory` / `donation_history`	Links a donor to an optional request and records donation date/type/location/notes.
`OrganDonationPreference` / `organ_donation_preference`	One preference per donor, with organs, consent text, verification flag and registration time.

Roles, blood groups and workflow statuses are Java enums. IDs are generated by persistence. The SQL schema is in `database/schema.sql`; the JPA entities are under `backend/src/main/java/com/smartdonation/entity`.

5. Hardcoded values and demo behavior

The code contains several fixed values intended for the local demo:

- The frontend API defaults to `http://localhost:8080/api`; it can be overridden with `window.SMART_DONATION_API_BASE` before loading the script.
- Blood-group options are a fixed list of the eight standard ABO/Rh groups in frontend JavaScript and a matching Java enum.

- The 90-day donation interval, distance score formula, maximum of 10 matches, earth radius constant and blood-compatibility map are coded rules.
- The development mail host is `localhost:2525`; the default configuration names Gmail SMTP but leaves username and password blank.
- The seeder uses a fixed password `password123` for demo accounts (stored as BCrypt hashes), sample names/emails, phone numbers, city and coordinates.
- Form convenience defaults include `A_POSITIVE` when a user has no blood group and a fixed demo phone/contact information in seeded records.

The hardcoded demo credentials are not suitable for real users or deployment. The application does not call an external maps service, SMS gateway, identity provider or production data service.

Development seed records

`StartupDataSeeder` runs only when the Spring profile is `dev`. It exits immediately if there is already at least one user. On an empty database it creates:

- Donor Priya Raman (`priya.donor@test.local`), O-negative, available, first-time/unknown donation date, organ donor.
- Donor Arun Kumar (`arun.donor@test.local`), A-positive, available, last donation 140 days ago.
- Donor Meera Shah (`meera.donor@test.local`), B-positive, available, last donation 35 days ago.
- Admin Officer (`admin@test.local`), City Hospital (`hospital@test.local`) and Ravi Patient (`patient@test.local`).
- One verified organ preference for Priya, and one open A-positive, two-unit emergency request for City Hospital in Chennai.

All six demo users share the password `password123`. The donor profiles and request have fixed example details, while relative donation dates use the date the app starts. The initial seeded request belongs to the seeded hospital account. Enter these credentials manually; quick-login shortcuts have been removed.

6. Does data come from the portal or seeding?

Both. The source of each record depends on how the app is started and what action is taken:

Data	Development profile (`run-dev.ps1`)	Default profile
Initial demo users, organ preference and blood request	Seeded into MySQL if there are no users	Not seeded by this seeder
New user registration	Created through portal/API in MySQL	Created through portal/API in MySQL
New blood requests	Created through portal/API in MySQL	Created through portal/API in MySQL
Organ preferences and donation history	Created through portal/API in MySQL	Created through portal/API in MySQL
Match and notification records	Generated by backend in MySQL when matching endpoint is called	Generated by backend in MySQL when matching endpoint is called
Persistence after application stops	Retained in MySQL	Retained in MySQL

The portal is not the only way to create data: the API can also be called directly. Conversely, the seed records are created by backend startup code rather than by a portal form. No SQL seed file is present in `database`; the sample records are in `StartupDataSeeder.java`.

Both profiles use the configured MySQL database with Hibernate schema update. On startup, the `dev` seeder checks for existing users and only inserts the demo dataset if the database is empty; it does not duplicate or top up seed rows

on subsequent restarts. The default profile has no startup demo seeder.

7. How to start the application

Recommended local demo: MySQL with the development profile

Prerequisites: Java 17, MySQL, Python, and Maven (a Maven distribution is supplied in `.tools`). Create `smart_donation_db`, set `DB_URL`, `DB_USERNAME` and `DB_PASSWORD`, then run from `backend`:

```
.\run-dev.ps1
```

The script launches `mvn spring-boot:run` with the `dev` profile. The backend API listens on `http://localhost:8080/api`. Demo seed data is written to the configured MySQL database and persists across restarts.

In another PowerShell window, open the `frontend` directory and run:

```
.\run.ps1
```

This starts Python's static file server on port 5501. Open `http://localhost:5501` in a browser; this is one of the configured CORS origins.

Demo login: enter one seeded account, such as `admin@test.local`, `hospital@test.local`, `patient@test.local`, or `priya.donor@test.local`, with password `password123`. To see matching, log in as hospital/patient and submit a request, or use the match button for a request owned by that account.

MySQL profile

Install and start MySQL, create the `smart_donation_db` database, then configure `DB_URL`, `DB_USERNAME` and `DB_PASSWORD` in the environment. Defaults are `jdbc:mysql://localhost:3306/smart_donation_db`, username `root`, and blank password. Start the backend from `backend` using:

```
mvn spring-boot:run
```

This uses the default profile, not `dev`; demo seeding is therefore disabled. Start the frontend separately as above. Real email delivery requires valid `SMTP_HOST`, `SMTP_PORT`, `SMTP_USERNAME` and `SMTP_PASSWORD` values.

8. API route reference

All routes are rooted at `http://localhost:8080/api`.

Method	Route	What it does
`GET`	`/auth/csrf`	Obtain a CSRF token for browser requests.
`POST`	`/auth/register`	Register a user and hash the password.
`POST`	`/auth/login`	Verify credentials and establish an HTTP session.
`GET`	`/auth/me`	Return the current authenticated profile.
`POST`	`/auth/logout`	Invalidate the current session.
`GET`	`/users`	List users (password hash is omitted from response DTO).
`POST`	`/blood-requests`	Create a blood request.
`GET`	`/blood-requests`	List blood requests.

Method	Route	What it does
POST	/blood-requests/{id}/match	Find eligible donors, create notifications and update request status.
POST	/blood-requests/{id}/fulfill	Mark a request fulfilled.
POST	/organs/preferences	Create/update an organ preference.
GET	/organs/preferences	List organ preferences.
POST	/organs/preferences/{id}/verify	Mark a preference verified.
POST	/donations	Record donation history and update donor's last donation date.
GET	/donations/donor/{donorId}	List donation history for a donor.
GET	/admin/dashboard	Get aggregate dashboard counts.

9. Security, limitations and production-readiness notes

- Authorization: protected routes require authentication. Only administrators can access the dashboard, user list, organ preference list and verification. Blood requests, donation histories and organ preferences enforce role permissions and ownership. Frontend navigation is not an access-control boundary.
- Authentication state: sessions are server-side; session IDs rotate on authentication; the `JSESSIONID` cookie is HttpOnly and SameSite=Lax; idle timeout defaults to 30 minutes. The base configuration defaults `APP_SESSION_COOKIE_SECURE=true`; the `dev` profile overrides it for local HTTP. Cross-site frontend/API deployments also need `APP_SESSION_COOKIE_SAME_SITE=None` over HTTPS. See [the security verification report](security-verification-report.md) for test evidence and deployment blockers.
- CSRF/CORS: state-changing requests require `X-XSRF-TOKEN`. CORS is credentialed and limited to configured frontend origins.
- Login throttling: the limiter is in-memory and local to one application instance. Multi-instance deployments need shared rate limiting or a trusted proxy/WAF limit.
- Validation: required-field checks are limited primarily to registration email/password and duplicate email. Request DTOs do not show comprehensive bean validation constraints.
- Email: SMTP configuration is incomplete locally. A notification row is still written if mail fails, with status pending. The request can still become `NOTIFIED`.
- Matching: rules are deterministic and illustrative. Medical eligibility and actual transfusion suitability must be determined by qualified services; the code should not be treated as clinical decision support. Coordinates and recommendation interval need validation against an approved operational process.
- Persistence: all profiles use MySQL, and `ddl-auto=update` keeps the demo convenient while retaining records across application restarts. It is not a controlled production migration strategy.
- Portal scope: the portal supports registration, login display, blood request submission and immediate matching, organ preference submission, and dashboard refresh. Donation-history operations are API-only in the current JavaScript.
- Configuration: database/SMTP credentials, CORS origins, session timeout, secure-cookie flag and login thresholds are environment-configurable. Demo credentials remain in the dev-only seeder and must not be used in a deployed environment.

10. Source files reviewed

- `README.md` — project overview and local run notes.
- `backend/pom.xml` — Java 17, Spring Boot 3.3.5, web, JPA, security, mail, MySQL and test dependencies.
- `backend/src/main/resources/application.properties` and `application-dev.properties` — database, mail and profile configuration.
- `backend/src/main/java/com/smartdonation/config/StartupDataSeeder.java` — development sample data.
- `backend/src/main/java/com/smartdonation/controller/` and `service/` — REST endpoints and workflows.
- `backend/src/main/java/com/smartdonation/matching/` — blood compatibility and distance logic.
- `backend/src/main/java/com/smartdonation/security/SecurityConfig.java` — endpoint access rules and password encoder.
- `backend/src/main/java/com/smartdonation/notification/NotificationService.java` — email attempt and notification persistence.
- `backend/src/main/java/com/smartdonation/entity/` and `database/schema.sql` — data model/schema.
- `frontend/index.html`, `frontend/assets/js/main.js`, `frontend/assets/css/styles.css`, and `frontend/run.ps1` — browser portal and local server.

Conclusion: The system accepts operational data through portal forms/API and includes a separate dev-only seed for demonstration. The local demo uses MySQL in both backend profiles; `backend\\run-dev.ps1` adds sample records to that persistent database, and `frontend\\run.ps1` serves the portal on port 5501.